Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА № 1

«Основные конструкции языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции конструкции языков программирования»

Выполнил: студент гр. ИУ5-33Б _____ (Ахметзянова Г.Т.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Оргсян О.Г.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель: изучение основных конструкций языка Python.

Задание: Разработать программы для решения биквадратного уравнения.

Выполнение работы.

1. Программа с применением процедурной парадигмы

```python
import sys
import math

def get_coef(index, prompt):
    while True:
        try:
            coef_str = sys.argv[index]
        except IndexError:
            print(prompt)
            coef_str = input()

        try:
            coef = float(coef_str)
            return coef
        except ValueError:
            print(f"Ошибка: '{coef_str}' не является числом. Попробуйте снова.")
            if len(sys.argv) > index:
                pass
                del sys.argv[index]

def get_roots(a, b, c):
    result = []
    D = b*b - 4*a*c
    if D < 0:
        return result
    sqD = math.sqrt(D)
    t_roots = []
    if D == 0:
        t_roots.append(-b / (2.0*a))
    else:
        t_roots.append((-b + sqD) / (2.0*a))
        t_roots.append((-b - sqD) / (2.0*a))
    for t in t_roots:
        if t > 0:
            root1 = math.sqrt(t)
            root2 = -math.sqrt(t)
            result.append(root1)
            result.append(root2)
        elif t == 0:
            result.append(0.0)
    return result

def main():
    a = get_coef(1, 'Введите коэффициент A:')
    b = get_coef(2, 'Введите коэффициент B:')
    c = get_coef(3, 'Введите коэффициент C:')
    roots = get_roots(a, b, c)
    len_roots = len(roots)
    if len_roots == 0:
        print('Нет корней')
    elif len_roots == 1:
        print('Один корень: {}'.format(roots[0]))
    elif len_roots == 2:
        print('Два корня: {} и {}'.format(roots[0], roots[1]))
    elif len_roots == 3:
        print('Три корня: {}, {} и {}'.format(roots[0], roots[1], roots[2]))
    elif len_roots == 4:
        print('Четыре корня: {}, {}, {} и {}'.format(roots[0], roots[1], roots[2], roots[3]))

if __name__ == "__main__":
    main()
```

Рисунок 1 – Код программы 1

```
Введите коэффициент A:
котик
Ошибка: 'котик' не является числом. Попробуйте снова.
Введите коэффициент A:
2
Введите коэффициент B:
3
Введите коэффициент C:
1
Нет корней
```

```
Введите коэффициент A:
1
Введите коэффициент B:
-5
Введите коэффициент C:
4
Четыре корня: 2.0, -2.0, 1.0 и -1.0
```

Рисунок 2 – Примеры работы программы 1

2. Программа с применением объектно-ориентированной парадигмы

```python
import sys
import math

class CoefficientReader:
    def __init__(self, args):
        self.args = args
    def get_coef(self, index, prompt):
        while True:
            try:
                coef_str = self.args[index]
            except IndexError:
                print(prompt)
                coef_str = input()
            try:
                return float(coef_str)
            except ValueError:
                print(f"Ошибка: '{coef_str}' не является числом. Попробуйте снова.")
                if len(self.args) > index:
                    del self.args[index]

class BiQuadraticEquation:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def solve(self):
        roots = []
        D = self.b**2 - 4 * self.a * self.c
        if D < 0:
            return roots
        sqD = math.sqrt(D)
        t_roots = []
        if D == 0:
            t_roots.append(-self.b / (2.0 * self.a))
        else:
            t_roots.append((-self.b + sqD) / (2.0 * self.a))
            t_roots.append((-self.b - sqD) / (2.0 * self.a))
        for t in t_roots:
            if t > 0:
                roots.append(math.sqrt(t))
                roots.append(-math.sqrt(t))
            elif t == 0:
                roots.append(0.0)
        return roots

def main():
    reader = CoefficientReader(sys.argv)
    a = reader.get_coef(1, 'Введите коэффициент A:')
    b = reader.get_coef(2, 'Введите коэффициент B:')
    c = reader.get_coef(3, 'Введите коэффициент C:')

    equation = BiQuadraticEquation(a, b, c)
    roots = equation.solve()

    len_roots = len(roots)
    if len_roots == 0:
        print('Нет корней')
    elif len_roots == 1:
        print('Один корень: {}'.format(roots[0]))
    elif len_roots == 2:
        print('Два корня: {} и {}'.format(roots[0], roots[1]))
    elif len_roots == 3:
        print('Три корня: {}, {} и {}'.format(roots[0], roots[1], roots[2]))
    elif len_roots == 4:
        print('Четыре корня: {}, {}, {} и {}'.format(roots[0], roots[1], roots[2], roots[3]))

if __name__ == "__main__":
    main()
```

Рисунок 3 – Код программы 2

```
Введите коэффициент A:
мяу
Ошибка: 'мяу' не является числом. Попробуйте снова.
Введите коэффициент A:
1
Введите коэффициент B:
6
Введите коэффициент C:
3
Нет корней
```

```
Введите коэффициент A:
1
Введите коэффициент B:
0
Введите коэффициент C:
-4
Два корня: 1.4142135623730951 и -1.4142135623730951
```

Рисунок 4 – Примеры работы программы 2

3. Программа с применением функциональной парадигмы

```python
import sys
import math

def parse_float(value):
    try:
        return float(value)
    except (ValueError, TypeError):
        return None

def get_input(args, index, prompt):
    val_str = args[index] if len(args) > index else None
    if val_str is None:
        print(prompt)
        val_str = input()
    parsed = parse_float(val_str)
    if parsed is None:
        print(f"Ошибка: '{val_str}' не является числом.")
        new_args = args[:index] + args[index+1:] if len(args) > index else args
        return get_input(new_args, index, prompt)
    return parsed, args

def solve_biquadratic(a, b, c):
    D = b**2 - 4*a*c
    if D < 0:
        return []
    sqD = math.sqrt(D)
    if D == 0:
        t_roots = [-b / (2.0*a)]
    else:
        t_roots = [(-b + sqD) / (2.0*a), (-b - sqD) / (2.0*a)]
    def t_to_x(t):
        if t > 0:
            return [math.sqrt(t), -math.sqrt(t)]
        elif t == 0:
            return [0.0]
        else:
            return []
    roots = []
    for t in t_roots:
        roots.extend(t_to_x(t))
    return roots

def format_output(roots):
    match roots:
        case []:
            return "Нет корней"
        case [x]:
            return f"Один корень: {x}"
        case [x, y]:
            return f"Два корня: {x} и {y}"
        case [x, y, z]:
            return f"Три корня: {x}, {y} и {z}"
        case [w, x, y, z]:
            return f"Четыре корня: {w}, {x}, {y} и {z}"
        case _:
            return "Неизвестное количество корней"

def main():
    args = sys.argv
    a, args = get_input(args, 1, 'Введите коэффициент A:')
    b, args = get_input(args, 2, 'Введите коэффициент B:')
    c, args = get_input(args, 3, 'Введите коэффициент C:')
    roots = solve_biquadratic(a, b, c)
    print(format_output(roots))

if __name__ == "__main__":
    main()
```

Рисунок 5 – Код программы 3

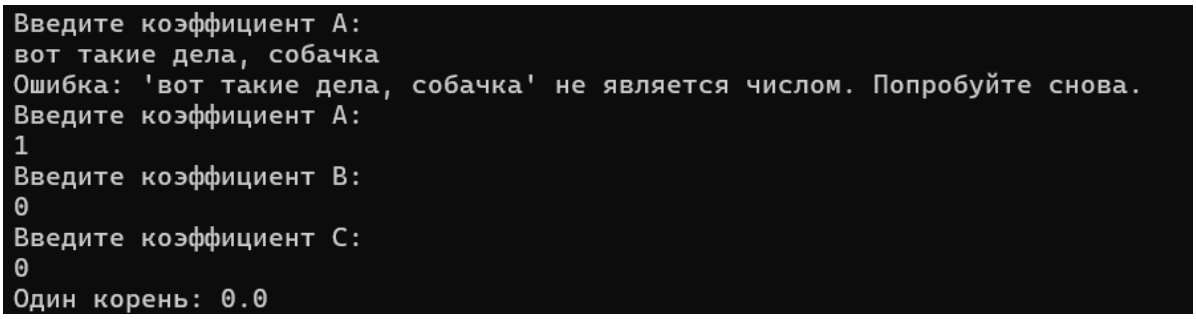


Рисунок 6 – Пример работы программы 3

Вывод: в результате выполнения работы были изучены основные конструкции языка Python.